CLAUDE CODE · INVENTARIO LOCAL

Manual de skills instaladas

Estado de `~/.claude/skills/` y `~/.claude/plugins/` en este equipo con Windows: ui-ux-pro-max y claude-mem documentados a fondo, cuatro skills de referencia que aún no están instaladas, y el resto del catálogo en un índice.

63	7	1	56
SKILLS TOTALES	UI-UX-PRO-MAX	PLUGIN (CLAUDE-MEM)	SUITE GSTACK + OTRAS

skills.sh · findskills-mcp · security-audit · ui-ux-pro-max · emil skills · claude-mem · brag · diagram-design · anatomy · rea · markitdown · freellmapi · supervision · playwright-mcp · firecrawl-mcp · perplexity-mcp · claude-video (watch) · 21st MCP · system_prompts_leaks · gstack · Referencia: ECC · Gestión de skills

- **Estado verificado el 17 oct 2026 en este PC con Windows:** hay 63 skills en `~/.claude/skills/` (las 7 de ui-ux-pro-max y la suite `gstack` v1.60.1.0) y el plugin `claude-mem` v13.28.0. `security-audit`, `brag`, `diagram-design` y `anatomy` están descritas abajo como referencia, pero **no están instaladas** en este equipo.

skills.sh

Directorio público de skills · CLI `skills` v1.7.2 (npm) · repo vercel-labs/skills

DIRECTORIO + CLI

QUÉ ES

skills.sh es el directorio público del ecosistema abierto de "agent skills": un catálogo y ranking donde se descubren skills por categoría y popularidad, con estadísticas de instalación. Detrás está la CLI `skills`, un proyecto MIT de Vercel Labs (`vercel-`

`labs/skills`) que instala las skills directamente desde repositorios Git. Muchas de las skills de este manual (por ejemplo `brag` y `diagram-design`) enlazan a su ficha en skills.sh.

Una skill es un conjunto reutilizable de instrucciones que amplía lo que puede hacer un agente de código. Vive en un directorio con un archivo `SKILL.md` cuyo encabezado YAML lleva un `name` y una `description`. El formato sigue la especificación abierta Agent Skills, por lo que una misma skill funciona en muchos agentes.

INSTALAR UNA SKILL

```
# todo un repositorio (owner/repo)
npx skills add vercel-labs/agent-skills

# ver qué trae antes de instalar
npx skills add vercel-labs/agent-skills --list

# una skill concreta, global, para Claude Code, sin preguntas
npx skills add vercel-labs/agent-skills --skill frontend-design -g -a claude-

# usar una skill sin instalarla (genera el prompt)
npx skills use vercel-labs/agent-skills@web-design-guidelines | claude
```

Fuentes admitidas: atajo de GitHub `owner/repo`, URL de GitHub o GitLab, Azure Repos, cualquier URL de git, una ruta local y la URL de descarga de un `SKILL.md` o de un archivo `.zip` / `.tar`. Los repos privados usan la autenticación que ya tengas configurada (credenciales de Git, GitHub CLI o SSH).

COMANDOS

COMANDO	QUÉ HACE
`npx skills add <fuente>`	Instala skills desde un repo, URL o ruta local.
`npx skills use <fuente>`	Usa una skill sin instalarla.
`npx skills find [consulta]`	Busca skills de forma interactiva o por palabra clave; `--owner` limita a un usuario u organización.
`npx skills list` (`ls`)	Lista las skills instaladas, con filtros `-g` y `-a`.
`npx skills update [skills]`	Actualiza a las últimas versiones (`-g` global, `-p` proyecto).
`npx skills remove [skills]` (`rm`)	Quita skills de uno o varios agentes.

`npx skills init [nombre]`	Crea una plantilla de `SKILL.md` para publicar la tuya.

OPCIONES DE `ADD`

OPCIÓN	QUÉ HACE
`-g, --global`	Instala en el directorio del usuario en vez del proyecto.
`-a, --agent`	Elige agentes concretos (por ejemplo `claude-code`, `codex`).
`-s, --skill`	Instala skills por nombre; `'*'` para todas.
`-l, --list`	Lista las skills disponibles sin instalar.
`--copy`	Copia archivos en lugar de usar enlaces simbólicos.
`-y, --yes` · `--all`	Sin confirmaciones · instala todo en todos los agentes.

DÓNDE SE INSTALAN

- **Proyecto** (por defecto): `./<agente>/skills/`. Se puede subir al repo y compartir con el equipo. Para Claude Code es `.claude/skills/`.
- **Global** (`-g`): `~/<agente>/skills/`. Disponible en todos tus proyectos; para Claude Code es `~/.claude/skills/`.
- Por defecto crea un enlace simbólico desde cada agente a una copia canónica, lo que facilita las actualizaciones. Con `--copy` cada agente tiene su copia independiente.
- La CLI detecta los agentes que tienes instalados. El README de la CLI lista unos 75 agentes compatibles, entre ellos Claude Code, Codex, Cursor, Gemini CLI, GitHub Copilot, OpenCode, Windsurf, Cline, Roo Code y Kiro CLI.

PUBLICAR TU PROPIA SKILL

1. Crea el directorio con `npx skills init mi-skill` y edita el `SKILL.md` (`name` y `description` son obligatorios).
2. Súbelo a un repositorio Git, en la raíz o bajo `skills/`. La CLI busca `SKILL.md` hasta tres niveles de profundidad.
3. Cualquiera lo instala con `npx skills add tu-usuario/tu-repo`. También puede declararse en `.claude-plugin/marketplace.json` para convivir con el marketplace de plugins de Claude Code.
4. Con `metadata.internal: true` la skill queda oculta, y solo se instala con `INSTALL_INTERNAL_SKILLS=1`.

COMPATIBILIDAD ENTRE AGENTES

Las skills básicas funcionan en todos los agentes de la tabla de compatibilidad. Algunas funciones son específicas: `allowed-tools` lo soportan casi todos (no Kiro CLI ni Zencoder), `context: fork` solo Claude Code, y los hooks Claude Code, Cline y Kiro CLI.

↗ skills.sh ↗ CLI en GitHub ↗ npm ↗ Especificación Agent Skills ↗ vercel-labs/agent-skills

- **Telemetría y seguridad:** la CLI envía datos de uso anónimos; los identificadores de repositorio y skill solo se mandan para repos que GitHub confirma como públicos. Se desactiva con `DISABLE_TELEMETRY=1` o `DO_NOT_TRACK=1`. Una skill es texto que el agente obedece y puede traer scripts, así que revisa el `SKILL.md` y su código antes de instalar la de un tercero. El ranking de popularidad no es una auditoría de seguridad.

- **Cómo se obtuvo esta información:** el sitio skills.sh no se pudo abrir desde el entorno donde se escribió esto (el proxy bloqueó la conexión), así que lo que aquí se dice de su catálogo, su ranking y sus estadísticas viene de resultados de búsqueda y de la documentación de Vercel resumida por terceros. Los comandos, opciones, rutas y la telemetría salen directamente del README de `vercel-labs/skills` (v1.7.2). Verifica en skills.sh el estado actual del catálogo y cómo se listan las skills nuevas.

findskills-mcp

Paquete npm `findskills-mcp` v0.1.25 · directorio findskills.org · MIT

NO INSTALADA · SERVIDOR MCP

QUÉ HACE

Servidor MCP para buscar skills de agentes desde Claude. Consulta el directorio FindSkills, que según el paquete reúne más de 93.000 skills. Es un buscador: devuelve fichas con calidad, seguridad y fuente, y no instala skills por sí mismo. Es una alternativa a skills.sh.

INSTALACIÓN

```
# asistente de un solo comando
npx -y findskills-mcp init
```

```
# manual, Claude Code
claude mcp add findskills -- npx -y findskills-mcp

# después, para obtener la clave (inicio de sesión con GitHub en el navegador)
npx findskills auth
```

`init` hace tres cosas: ejecuta `npm install -g findskills-mcp`, busca la ruta absoluta del binario global y escribe una entrada `findskills` (tipo stdio) en `~/.claude.json`; si ese archivo no existe pero sí el de Claude Desktop en macOS, edita ese. No autentica por sí solo. Al terminar hay que reiniciar Claude Code o Claude Desktop.

`↗ findskills.org` `↗ npm`

HERRAMIENTAS

HERRAMIENTA	QUÉ HACE
`search_skills`	Busca por palabra clave; ordena por relevancia, calidad, seguridad, estrellas, nombre o fecha.
`list_skills`	Navega y filtra por categoría, fuente, tipo de autenticación y puntajes mínimos de calidad y seguridad (0 a 100), con paginación.
`get_skill`	Detalle de una skill. Se le pasa el `id` de la búsqueda, no el nombre.
`get_stats`	Resumen del dataset: total y distribuciones. Se recomienda llamarla primero.
`list_tags`	Etiquetas con su número de usos.

Uso típico: *"busca skills de web scraping con buena calidad y seguridad"*.

QUÉ NECESITA

- Node.js 20 o superior (el manual indica Node v24.18.0 en este equipo).
- Una clave de API gratuita de FindSkills, que se consigue con `npx findskills auth` y se guarda en `~/.findskills/config.json`. Sin ella las llamadas devuelven 401.
- Permiso para que npm instale paquetes globales, y acceso de red a `findskills.org`.

- **Problemas probables en la v0.1.25:** leí el código del paquete sin ejecutarlo y encontré dos fallos. (1) `api.js` importa `./lib/auth-error.js`, que no viene en el paquete, así que el servidor probablemente falla al cargar. (2) La clave se lee mediante `../cli/lib/config.js`, que tampoco viene incluido, de modo que aunque te autentiques las llamadas pueden salir sin clave. El paquete hermano `findskills` va en la 0.2.5, más reciente. Antes de instalarlo conviene probar `npx -y findskills-mcp`

`--version` y una consulta real, o buscar una versión corregida. Además, `init` instala un paquete global y reescribe `~/.claude.json`: haz una copia antes. No revisé el paquete `findskills` ni el sitio findskills.org.

security-audit

Destino previsto: `~/.claude/skills/security-audit/`

NO INSTALADA · 1 SKILL · CLOUDFLARE

QUÉ HACE

Convierte al agente en auditor de seguridad. Orquesta agentes aislados a lo largo de seis fases: reconocimiento, caza guiada por cobertura, validación de candidatos, salida estructurada (`findings.json`), verificación independiente de los registros y reporte neutral (`REPORT.md`, `FINDINGS-DETAIL.md`, `NEEDS-VALIDATION.md`). Es la skill que dio origen al arnés de descubrimiento de vulnerabilidades de Cloudflare.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# clonado desde
git clone https://github.com/cloudflare/security-audit-skill

# copiado skills/security-audit/ a
~/.claude/skills/security-audit/

# alternativa oficial (Skills CLI) no usada aquí
npx skills add https://github.com/cloudflare/security-audit-skill \
  --skill security-audit --global
```

↗ Repositorio en GitHub ↗ Blog de Cloudflare

CÓMO SE USA

- Pedidos como `security audit this codebase`, `find security vulnerabilities in ./src` o `do a security review, output to ~/audits/mi-proyecto`.
- **Modo guía** (por defecto): preguntas de seguridad o revisiones puntuales, sin crear archivos.
- **Modo auditoría completa**: solo si pides auditar o hacer pen-test de un repo, o reportes. Salida por defecto en `~/security-audit-skill/<repo>/run-N`.

→ Varias ejecuciones sobre el mismo repo son aditivas: una sola corrida encontró aprox. la mitad de lo que encontraron las repetidas.

REQUISITOS

Node.js para los validadores (`validate-findings.cjs` , `validate-coverage-ledger.cjs` ; sin dependencias). Aquí hay Node `v24.18.0` . Para ejecutar código del objetivo exige un sandbox del SO sin red; sin él, deja el hallazgo como `needs_validation` en vez de ejecutar nada.

ui-ux-pro-max

`~/.claude/skills/{ui-ux-pro-max, design, design-system, ui-styling, brand, banner-design, slides}/`

7 SKILLS

QUÉ HACE

Inteligencia de diseño UI/UX searchable en local: 79 estilos visuales (50 activos), 192 paletas de color con perfil de razonamiento por tipo de producto, 74 combinaciones tipográficas, 119 guías de UX, 105 iconos curados, 17 presets de GSAP, 25 tipos de gráfico y 22 stacks técnicos (React, Next.js, Vue, Nuxt, Svelte, Astro, SwiftUI, Flutter, Tailwind, shadcn/ui, Jetpack Compose, Angular, Laravel, Three.js, y más).

El plugin trae siete skills relacionadas, no solo una. El repo publica el paquete completo bajo `.claude/skills/` , así que instalar "el skill" instala las siete.

INSTALACIÓN (YA APLICADA)

```
# clonado desde
git clone https://github.com/nextlevelbuilder/ui-ux-pro-max-skill.git

# copiados los 7 directorios de .claude/skills/ del repo a
~/.claude/skills/ui-ux-pro-max/
~/.claude/skills/design/
~/.claude/skills/design-system/
~/.claude/skills/ui-styling/
~/.claude/skills/brand/
~/.claude/skills/banner-design/
~/.claude/skills/slides/

# alternativas oficiales no usadas aquí
```

```
/plugin marketplace add nextlevelbuilder/ui-ux-pro-max-skill
/plugin install ui-ux-pro-max@ui-ux-pro-max-skill

npx ui-ux-pro-max-cli init --ai claude --global
```

↗ Repositorio en GitHub ↗ uupm.cc

- Se copiaron los archivos estáticos directo del repo en vez de correr `npx ui-ux-pro-max-cli` — evita ejecutar el instalador npm de un tercero; el resultado en disco es el mismo.

LAS 7 SKILLS DEL PAQUETE

ui-ux-pro-max
Motor principal: búsqueda de estilos, paletas, tipografía, charts y reglas UX/accesibilidad por prioridad.

design
Identidad de marca, tokens, logos (55 estilos), CIP, banners, iconos, fotos sociales.

design-system
Arquitectura de tokens en tres capas (primitivo→semántico→componente) y specs de componentes.

ui-styling
Componentes accesibles con shadcn/ui (Radix + Tailwind) y diseños basados en canvas.

brand
Voz de marca, mensajería, gestión de assets y consistencia de guías de estilo.

banner-design
Banners para redes, ads y hero de sitio — 22 estilos, múltiples formatos.

slides
Presentaciones HTML con Chart.js, tokens de diseño y fórmulas de copywriting.

CÓMO SE DISPARA

→ Automático al diseñar, construir o revisar cualquier UI: páginas, componentes, paletas, tipografía, layout, accesibilidad, animación.

→ También responde a pedidos explícitos de logo, banner, CIP, icono o presentación HTML.

→ Se salta en lógica de backend, APIs, infraestructura o scripts sin impacto visual.

REQUISITO

Python 3.x para el script de búsqueda (solo librería estándar, no instala nada ni hace llamadas de red). Ya está presente en esta máquina: `Python 3.14.6`.

emilkowalski/skills

`~/.claude/skills/{emil-design-eng, animate, review-animations, …}/ · repo emilkowalski/skills · MIT`

NO INSTALADA · 15 SKILLS

QUÉ HACE

Paquete de 15 skills para diseñadores e ingenieros que construyen interfaces, escrito por Emil Kowalski a partir de su experiencia en empresas como Vercel y Linear. Parte de una idea: los agentes no tienen buen gusto y cometen errores pequeños que se acumulan, como usar `ease-in` en una animación de entrada cuando debería ser `ease-out`, o un borde sólido en vez de una sombra semitransparente. Las skills listan esos errores y explican cómo corregirlos.

El contenido es solo Markdown (`SKILL.md` y algunos archivos de apoyo): sin scripts ni código ejecutable. El repo incluye además una `performance-cheatsheet.md` con reglas rápidas, como animar `transform` y `opacity` en lugar de `width` o `top` y no usar `transition: all`.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# con la CLI de skills.sh
npx skills@latest add emilkowalski/skills

# solo una skill, global, para Claude Code
npx skills add emilkowalski/skills --skill animate -g -a claude-code
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Agents with Taste ↗ Newsletter

BASE Y CRITERIO

SKILL	QUÉ HACE
emil-design-eng	La skill principal: la filosofía del autor sobre pulido de UI, diseño de componentes y decisiones de animación.

`animation-vocabulary`	Enseña a pedir animaciones con las palabras exactas para que el agente entienda lo que quieres.
`apple-design`	Principios de diseño y movimiento fluido de Apple, extraídos de sus charlas de WWDC y traducidos a la web.
`mobile-native`	Hace que una web se sienta nativa en el teléfono: hover pegajoso, destello al tocar, bug de 100vh, zoom en inputs, áreas seguras.
`write-swift`	Swift moderno: tipos de valor, concurrencia de Swift 6, genéricos, rendimiento y Swift Testing.
`pick-ui-library`	Elige la librería adecuada entre las que el autor usa y confía, en vez de inventar un toast o instalar un paquete abandonado.
`ask-sonner`	Guía para Sonner, la librería de toasts del autor: configuración, estilos, recetas y problemas comunes.

CREAR

SKILL	QUÉ HACE
`animate`	Construye una animación desde cero decidiendo en orden: si debe animarse, con qué propósito, herramienta, propiedades, curva y duración, interrupción y salida.
`animate-expo`	Lo mismo para React Native y Expo: gestos, sheets, háptica, transiciones de pantalla y movimiento fuera del hilo de JS.
`prototype`	Genera varias versiones distintas de una pieza de UI y las muestra con un selector para elegir. Solo corre si la invocas tú.

REVISAR Y PROBAR

SKILL	QUÉ HACE
`review-animations`	Revisa tus animaciones con las reglas del autor, de forma estricta.
`improve-animations`	Audita todas las animaciones del código y entrega planes priorizados que cualquier agente puede ejecutar.

`find-animation-opportunities`	Busca dónde el movimiento aportaría de verdad, y qué no conviene animar.
`break-ui`	Prueba la UI con datos extremos (nombres largos, listas vacías, contadores enormes, emoji) y reporta lo que se rompe con su arreglo.
`animation-performance`	Arregla animaciones que pierden fotogramas, con problemas sutiles que muchos agentes pasan por alto.

CÓMO SE DISPARA

- Cada skill se activa por su descripción (por ejemplo, `animate` cuando pides añadir movimiento o una transición) o con `/nombre`. `prototype` tiene `disable-model-invocation`: solo corre si la invocas tú.
- Para criticar movimiento existente se usa `review-animations`; para auditar todo el código, `improve-animations`; para crítica visual general, `emil-design-eng`.

- **A tener en cuenta:** las skills son largas (`write-swift` ronda los 42 KB y `emil-design-eng` los 27 KB), así que cargan bastante contexto al activarse. `emil-design-eng` y `apple-design` se solapan en parte con `ui-ux-pro-max` y con `design-review` de gstack; conviene decidir cuál manda al pedir "mejorar la UI". Es una guía de criterio, no un generador de diseños: rinde más sobre una UI ya hecha. Datos tomados del README y de los encabezados de los `SKILL.md`; no instalé ni ejecuté las skills.

claude-mem

`~/.claude/plugins/marketplaces/thedotmack/ · cache en .../cache/thedotmack/claude-mem/13.28.0`

PLUGIN + WORKER

QUÉ HACE

Sistema de memoria persistente para Claude Code. Captura observaciones de uso de herramientas durante la sesión, genera resúmenes semánticos y los deja disponibles para sesiones futuras del mismo proyecto — sin que el usuario tenga que repetir contexto.

A diferencia de ui-ux-pro-max, no es un conjunto de archivos estáticos: registra hooks de ciclo de vida en Claude Code y corre un worker HTTP en segundo plano.

INSTALACIÓN Y CONFIGURACIÓN (INSTALADO EL 27 SEP 2026)

```
# instalado sin cuenta / sin nube, con el plan de Anthropic ya logueado
CLAUDE_MEM_ONLINE_OPTIN=false npx claude-mem@latest install \
  --ide claude-code --provider claude

# telemetría anónima apagada
npx claude-mem telemetry disable

# worker local iniciado manualmente (el installer lo dejó sin arrancar)
npx claude-mem start
```

↗ Repositorio en GitHub ↗ Web viewer del worker

ESTADO ACTUAL

CONFIG	VALOR
Proveedor	`claude` — usa la sesión de Anthropic ya logueada, sin API key aparte
Cloud sync	OFF — nada se manda a cmem.ai salvo el contacto único de alta si te logueas
Telemetría	OFF — deshabilitada explícitamente tras instalar
Worker	Corriendo en `http://127.0.0.1:37777`
Datos	Todo local en `~/.claude-mem/`

- El worker responde en `http://127.0.0.1:37777` (estado *ok*, v13.28.0). La memoria se acumula al usar Claude Code en un proyecto y se inyecta desde la segunda sesión de ese proyecto.

SKILL PRINCIPAL + COMANDOS

El plugin expone `mem-search` como la skill de cara al usuario (búsqueda de observaciones pasadas), más ~19 skills internas de soporte (`learn-codebase`, `how-it-works`, `knowledge-agent`, `standup`, `agent-cost-report`, `weekly-digests`, entre otras) para flujos específicos del propio proyecto claude-mem.

COMANDO	QUÉ HACE

`npx claude-mem status`	Estado del worker (PID, puerto, uptime, dependencias)
`npx claude-mem doctor`	Diagnóstico de runtime (Bun, uv, worker, plugin)
`npx claude-mem start / stop / restart`	Control manual del worker en segundo plano
`npx claude-mem search "<query>"`	Busca observaciones guardadas desde la terminal
`npx claude-mem telemetry status`	Confirma que la telemetría sigue apagada
`npx claude-mem uninstall`	Quita el plugin, hooks y configuración

brag

NO INSTALADA · 1 SKILL · VIDEO

Destino previsto: ~/.claude/skills/brag/

QUÉ HACE

Convierte el proyecto actual en un video corto de lanzamiento, con música, movimiento y copy para compartir. Lee el código del proyecto directamente, sin necesitar URL en vivo ni capturas de pantalla. La composición del video la ejecuta Hyperframes; la skill decide el ángulo de producto, el tono y qué momentos mostrar.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# clonado desde
git clone https://github.com/latent-spaces/brag.git

# copiado skills/brag/ a
~/.claude/skills/brag/
  ├── SKILL.md
  ├── assets/    # música y efectos de sonido incluidos
  ├── references/
  └── scripts/

# alternativa con el instalador oficial (cualquier agente)
npx skills add https://github.com/latent-spaces/brag --skill brag

# o como plugin nativo de Claude Code
```

```
/plugin marketplace add latent-spaces/brag
/plugin install brag@brag
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Hyperframes

REQUISITOS PARA EJECUTARLA

- → Node.js 22+
- → FFmpeg disponible en `PATH`
- → CLI de Hyperframes; verifícalo con `npx hyperframes doctor`

USO BÁSICO

1. Párate en la carpeta del proyecto que quieres mostrar.
2. Escribe `let's /brag` o simplemente `/brag`.
3. La skill lee el código y arma un guion y un storyboard cortos.
4. Entrega un brief de composición a Hyperframes, que renderiza el video.
5. Recibes una carpeta `brag-output/` con plan, brief, copy para redes y `brag.mp4`.

OPCIONES (FLAGS)

FLAG	VALORES	POR DEFECTO
`--tone`	preset o descripción libre	inferido
`--format`	`landscape` / `vertical` / `square`	`landscape`
`--duration`	segundos	auto (15–25 s)
`--no-music`	flag	música activada
`--no-sfx`	flag	efectos activados
`--title`	texto	inferido del proyecto
`--voice`	flag	narración apagada

Los presets de tono son `default`, `polished`, `yc-parody`, `chaotic`, `deadpan`, `cinematic` y `app-store`. También acepta dirección libre:

```
/brag --tone "fake Series A launch from 2016"
```

```
/brag --voice
```

- La narración usa Kokoro vía Hyperframes y solo se activa con `--voice`. Los requisitos (FFmpeg, Hyperframes) no se han verificado en esta máquina.

diagram-design

Destino previsto: ~/.claude/skills/diagram-design/

NO INSTALADA · 1 SKILL · 45 TIPOS

QUÉ HACE

Dibuja diagramas como un único archivo HTML con SVG incrustado, con estilo editorial y sin sombras. Cubre 45 tipos: arquitectura, flujo, secuencia, máquina de estados, ER, línea de tiempo, swimlane, cuadrante, radar, Sankey, Gantt, mapa de Wardley, kanban, UML, esquema de base de datos y más. El agente elige el tipo, explica su plan y escribe el archivo, que se abre con doble clic.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# clonado desde
git clone https://github.com/cathrynlavery/diagram-design.git

# copiado a
~/.claude/skills/diagram-design/
  ├── SKILL.md
  ├── assets/
  ├── references/
  └── scripts/   # importadores y exportador, solo Python local

# alternativa con el instalador oficial (cualquier agente)
npx skills add cathrynlavery/diagram-design

# o como plugin de Claude Code (añade comandos slash)
/plugin marketplace add cathrynlavery/diagram-design
/plugin install diagram-design@diagram-design
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Sitio web ↗ Galería

USO BÁSICO

1. Pide el diagrama en lenguaje natural, por ejemplo `Hazme un diagrama de arquitectura: frontend, backend, base de datos y caché Redis`.
2. La primera vez en un proyecto, pregunta si quieres adaptar el estilo a tu marca: URL, skill instalada, carpeta, tokens pegados, estilo por defecto o perfil guardado.
3. El agente indica qué tipo de diagrama va a usar y escribe un archivo `.html`.
4. Abre el archivo en el navegador, o pide exportarlo a PNG o SVG.

IMPORTAR Y EXPORTAR

- → Importa archivos `.drawio`, Mermaid y `.excalidraw` y los redibuja en tu estilo.
- → Exporta a PNG o SVG con `/export-diagram` (versión plugin).
- → `/doctor` revisa que la instalación esté completa (versión plugin).

- Sin servidor, cuentas ni telemetría. Lo único que sale a internet son las fuentes de Google Fonts cuando abres un diagrama en el navegador.

anatomy

NO INSTALADA · 1 SKILL · FIGURAS

Destino previsto: ~/.claude/skills/anatomy/

QUÉ HACE

Explica una idea técnica construyéndola como un objeto físico que funciona. Una clave de caché que evita trabajo repetido se vuelve una cerradura de pines que solo gira si todos los pines coinciden. Genera figuras isométricas interactivas en SVG, con muchas piezas pequeñas sombreadas, dentro de una tarjeta con título, pista y lectura en vivo. Puede añadir shaders WebGL (fuego, agua, luz) dentro del mismo dibujo.

Cada número que muestra la figura sale del mismo modelo que mueve sus piezas. Funciona sin framework (TypeScript/JS que escribe SVG y HTML) y en React/Next.js.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# el repositorio completo es la carpeta de la skill
git clone https://github.com/wheresryan22/anatomy.git
# copiado (sin .git) a
~/.claude/skills/anatomy/
```

```
# alternativa con el instalador oficial
npx skills add wheresryan22/anatomy -g -a claude-code

# otros agentes: cambia -a por opencode o codex
```

↗ Repositorio en GitHub ↗ Sitio web ↗ Descargar ZIP

REQUISITOS

- Node.js 20.10 o superior. Aquí hay Node `v24.18.0`.
- Chrome o Chromium para las comprobaciones de la figura. No verificado en esta máquina.
- Sin API keys ni `npm install`: usa solo módulos integrados de Node.

USO

```
/anatomy explain how a token bucket rate limiter works, interactive
```

- También se activa solo ante pedidos de ilustraciones isométricas, figuras explicativas para docs o blogs, o SVG que muestren cómo funciona un sistema.
- El repo incluye 8 ejemplos en `examples/` (motor Raptor, tanque de ondas, indicador de dial, máquina arcade y otros).
- Puede publicar una figura terminada en la galería comunitaria, solo si se lo pides.

- Los scripts de captura solo lanzan Chrome local y se comunican con `127.0.0.1`. No hay llamadas a servicios externos en el código de la skill.

rea · Reverse Engineer Anything

Destino previsto: MCP + skill `reverse-engineer-anything` · paquete npm `rea-agents` v6.5.0

NO INSTALADA · MCP + 1 SKILL

QUÉ HACE

Un solo servidor MCP para hacer ingeniería inversa de binarios, aplicaciones y comportamiento en ejecución. Ves una función en una app, le pides al agente que investigue cómo funciona con REA y te devuelve la explicación con la evidencia y sus límites, sin

necesitar el código fuente. El análisis corre en local y también se puede usar desde la terminal.

Cubre binarios nativos (con Hopper, Ghidra o IDA), JavaScript/Electron, ensamblados .NET, APK de Android, sitios web, firmware, capturas de red y de paquetes, bases SQLite, WASM y bytecode EVM, entre otros.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# registra el MCP y las instrucciones de flujo en tu agente
npx rea-agents setup

# CLI para uso normal
npm install --global rea-agents
rea --help

# ejemplo desde la terminal: app JavaScript/Electron extraída o ASAR
npx -y rea-agents@latest analyze-javascript-application /ruta/absoluta/app --
```

Setup muestra los cambios, pide aprobación, guarda copias de la configuración existente y soporta Claude Code, Codex, Cursor, Gemini CLI y otros agentes. Reinicia el agente después.

↗ Repositorio en GitHub ↗ rea.tools ↗ Ver en skills.sh ↗ npm

CÓMO SE USA

```
Understand how search works in the Notes app, show me the evidence,
and build a similar feature for my project.
```

- La skill `reverse-engineer-anything` se activa cuando una conclusión depende de un binario o paquete ya distribuido, de decompilación, de evidencia pasiva en ejecución o de comparar versiones. Se salta el análisis de arquitectura de un repo con código fuente.
- El repo trae además 4 skills de mantenimiento propias del proyecto en `.agents/skills/` (`rea-tool-design` , `rea-analysis-optimization` , `rea-ci-tests-audit` , `rea-changelog-update`); no hacen falta para usar REA.

REQUISITOS

- Node.js 22.x (≥22.19), 24.x (≥24.11) o 26+, más npm.
- Binarios nativos: Hopper, Ghidra o IDA ya instalados. Android: JADX y un JDK completo. Sitios web: un navegador de la familia Chrome.
- En Linux, algunos análisis piden un `ps` compatible con procps en el `PATH` .

- Licencia MIT; el repo incluye un `DISCLAIMER.md` y `SECURITY.md`. Descárgalo solo desde github.com/morluto/rea, rea.tools o el paquete npm `rea-agents`: el propio README advierte que un nombre parecido no garantiza que sea oficial. La captura en ejecución usa tus permisos de usuario, así que revisa la guía de cada modo antes de analizar software ajeno. Datos tomados del README; no instalé ni ejecuté REA.

markitdown

Paquete PyPI `markitdown` v0.1.8 · repo microsoft/markitdown

NO INSTALADA · CLI + MCP

QUÉ HACE

Utilidad ligera de Python (de Microsoft) que convierte archivos a Markdown para usarlos con LLMs y flujos de análisis de texto. Conserva la estructura importante (títulos, listas, tablas, enlaces). La salida está pensada para que la lean herramientas de análisis, así que no es la mejor opción para conversiones de alta fidelidad destinadas a personas.

No es una skill ni un `SKILL.md`: es una herramienta de línea de comandos y una librería. Se incluye aquí porque se puede conectar a un agente como servidor MCP.

Formatos: PDF, PowerPoint, Word, Excel, imágenes (EXIF y OCR), audio (EXIF y transcripción), HTML, CSV/JSON/XML, ZIP (recorre el contenido), URLs de YouTube, EPub y más.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# Python 3.10 a 3.14; conviene usar un entorno virtual
python -m venv .venv
source .venv/bin/activate
pip install 'markitdown[all]'

# solo algunos formatos
pip install 'markitdown[pdf, docx, pptx]'
```

↗ Repositorio en GitHub ↗ PyPI ↗ markitdown-mcp

USO

```
markitdown archivo.pdf > documento.md
markitdown archivo.pdf -o documento.md
cat archivo.pdf | markitdown
```

OPCIÓN	QUÉ HACE
`[all]`	Instala todas las dependencias opcionales.
`[pdf]` `[docx]` `[pptx]` `[xlsx]` `[xls]`	Dependencias por formato de documento.
`[outlook]`	Mensajes de Outlook.
`[audio-transcription]`	Transcripción de audio wav y mp3.
`[youtube-transcription]`	Transcripción de videos de YouTube.
`[az-doc-intel]` `[az-content-understanding]`	Conversión en la nube con Azure; son llamadas de pago.
`--use-plugins` · `--list-plugins`	Activa o lista plugins de terceros (desactivados por defecto).

PAQUETES DEL REPO

`markitdown`
Librería y CLI principal.

`markitdown-mcp`
Servidor MCP (STDIO, HTTP y SSE) con una herramienta: `convert_to_markdown(uri)` .

`markitdown-ocr`
Plugin de OCR con visión de LLM para PDF, DOCX, PPTX y XLSX.

COMO SERVIDOR MCP

```
pip install markitdown-mcp
markitdown-mcp                                    # STDIO (por defecto)
markitdown-mcp --http --host 127.0.0.1 --port 3001
```

Acepta URIs `http:` , `https:` , `file:` y `data:` . Para Claude Desktop el README recomienda la imagen de Docker; para leer archivos locales hay que montar el directorio, por ejemplo `-v /home/user/data:/workdir` .

- **Seguridad:** MarkItDown lee archivos y URLs con los privilegios del proceso que lo ejecuta, como `open()` o `requests.get()` . Con entradas no confiables, usa la función `convert_*` más estrecha posible. El servidor MCP es solo para uso local:

escucha en `localhost` por defecto, y no conviene enlazarlo a otras interfaces. Datos tomados del README; no instalé ni ejecuté la herramienta.

freellmapi

Repo tashfeenahmed/freellmapi · MIT · servidor local en `http://localhost:3001`

NO INSTALADA · HERRAMIENTA

QUÉ HACE

Aplicación que junta los planes gratuitos de decenas de proveedores de IA detrás de un único endpoint `/v1` compatible con OpenAI. El README anuncia 34 proveedores y 635 endpoints de modelos gratuitos, unos 7.400 millones de tokens al mes en total, entre ellos Google, Groq, Cerebras, Mistral, OpenRouter, Cloudflare, Cohere, NVIDIA y HuggingFace. También admite tu propio endpoint compatible (llama.cpp, LM Studio, vLLM, Ollama).

No es una skill ni trae `SKILL.md`: es un servidor que se instala aparte. Se incluye aquí porque se puede conectar a agentes de código como Claude Code, Codex CLI, Gemini CLI, Cursor, Cline, Roo Code, Aider y OpenCode.

CÓMO FUNCIONA

- Agregas tus propias claves de cada proveedor en la página **Keys**; se guardan cifradas (AES-256-GCM).
- Un enrutador elige el mejor modelo disponible para cada petición, pasa al siguiente proveedor si uno llega a su límite (429/5xx) y cuenta el uso por clave para no superar ningún tope gratuito.
- Tus aplicaciones solo ven una clave unificada `freellmapi-…`.
- Admite chat, `/v1/responses`, embeddings, imágenes, audio y video, la API de mensajes de Anthropic (`/v1/messages`), una superficie nativa de Gemini y emulación opcional de Ollama.
- Extras: modelo virtual `fusion` (varios modelos responden y uno sintetiza), perfiles de cadena de respaldo, compresión de prompts opcional y un servidor MCP en `/mcp`.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# requiere Docker; genera la clave de cifrado y levanta el contenedor
curl -fsSL https://freellmapi.co/install.sh | bash
```

Después abre `http://localhost:3001`, agrega tus claves en **Keys**, ordena la **Fallback Chain** y copia la clave unificada. Hay también instaladores de escritorio para macOS y Windows, apps móviles y guías de Docker Compose, desarrollo local y Termux en Android. Funciona donde corra Node 20+.

↗ Repositorio en GitHub · ↗ freellmapi.co · ↗ Script de instalación

CONECTARLO A UN AGENTE

```
# configura Claude Code: lee el catálogo, respalda tu config y la fusiona
npx freellmapi setup-claude --url http://localhost:3001 --api-key <clave-unif
```

AGENTE	COMANDO
Claude Code	`setup-claude` o `launch`
Codex CLI	`setup-codex` o `launch-codex`
Cline · Roo Code · Continue	`setup-cline` · `setup-roo` · `setup-continue`
Aider · OpenCode	`setup-aider` · `setup-opencode`

Para cualquier otro cliente OpenAI basta con apuntar la URL base a `http://localhost:3001/v1` y usar la clave unificada.

- **Antes de usarlo:** el propio README lo declara para experimentación personal, *no para producción*: no hay modelos de frontera, la latencia varía, no hay SLA y la capacidad baja a última hora del día cuando los mejores modelos agotan su tope diario, que se restablece a medianoche UTC. El tráfico sigue sujeto a los términos de cada proveedor, y eso es responsabilidad tuya. El catálogo se actualiza solo desde freellmapi.co; las instalaciones gratuitas lo reciben con 30 días de retraso y la versión premium (19 USD al año) el mismo día. El comando de instalación descarga y ejecuta un script remoto: léelo antes. Datos tomados del README; no instalé ni ejecuté la herramienta.

supervision

Paquete PyPI `supervision` v0.31.0 · repo roboflow/supervision · MIT

SOLO MENCIONADA · LIBRERÍA PYTHON

QUÉ HACE

Caja de herramientas de visión por computador de Roboflow. No carga ni entrena modelos: trabaja con las detecciones de cualquier modelo de clasificación, detección o segmentación (RF-DETR, YOLO, Inference y otros, mediante conectores) y ofrece los bloques para construir aplicaciones alrededor de ellos.

- **Anotadores:** cajas, máscaras y otros, componibles, para dibujar las detecciones sobre imágenes o video.
- **Conjuntos de datos:** cargar, dividir, fusionar y guardar datasets en formatos como COCO y YOLO.
- **Utilidades de video y análisis:** conteo por zonas en tiempo real, seguimiento, geometría, puntos clave y métricas.

No es una skill ni un servidor MCP: es una librería de Python.

INSTALACIÓN (SOLO REFERENCIA, NO INSTALADA)

```
# Python 3.10 o superior, mejor en un entorno virtual
pip install supervision

# ejemplo del README: modelo RF-DETR (dependencias opcionales)
pip install pillow rfdetr
```

↗ Repositorio en GitHub ↗ Documentación ↗ PyPI

QUÉ FALTA PARA PODER INSTALARLA

- **Python ≥ 3.10.** El manual indica Python 3.14.6 en este equipo; el repo declara `⩾3.10`, pero no verifiqué que todas sus dependencias (numpy, scipy, matplotlib, av, entre otras) tengan paquetes para 3.14.
- Un entorno virtual (`python -m venv .venv`) para no mezclar dependencias.
- **Un modelo de visión** que genere las detecciones: `rfdetr`, `ultralytics`, `inference` u otro. La librería sola no detecta nada.
- Para `inference`, una **API key de Roboflow**. Para descargar datasets de Roboflow, la cuenta y el ID del proyecto.
- Un equipo con GPU conviene para video en tiempo real; no es obligatorio para probar con imágenes.

USO

```
import supervision as sv
```

```
detections = sv.Detections.from_inference(result)   # o from_ultralytics, etc
annotated = sv.BoxAnnotator().annotate(scene=image.copy(), detections=detecti
```

- Datos tomados del README y de `pyproject.toml` del repo; no instalé ni ejecuté la librería.

Playwright MCP

Paquete npm `@playwright/mcp` v0.0.83 · repo microsoft/playwright-mcp

SOLO MENCIONADA · SERVIDOR MCP

QUÉ HACE

Servidor MCP de Microsoft que da a un agente control de un navegador real con Playwright. En lugar de capturas de pantalla, usa el árbol de accesibilidad de la página, así que no necesita un modelo de visión y las acciones son más deterministas. Sirve para navegar, hacer clic, rellenar formularios y leer páginas.

El propio README aclara que para **agentes de código** puede ser más eficiente en tokens usar `playwright-cli` con skills; el servidor MCP queda para flujos con estado persistente, exploración y pruebas autorreparables.

INSTALACIÓN (SOLO REFERENCIA, NO INSTALADA)

```
# Claude Code
claude mcp add playwright npx @playwright/mcp@latest

# configuración estándar para otros clientes MCP
{ "mcpServers": { "playwright": { "command": "npx", "args": ["@playwright/mcp(
```

↗ Repositorio en GitHub ↗ Guía en playwright.dev ↗ npm

QUÉ FALTA PARA PODER INSTALARLA

→ **Node.js 18 o superior** (el manual indica Node v24.18.0 en este equipo).

→ **Un navegador**: Chrome por defecto; con `--browser` se puede elegir `chrome`, `firefox`, `webkit` o `msedge`. En Docker solo hay Chromium sin interfaz.

→ **Un cliente MCP** donde registrarlo (Claude Code, Cursor, VS Code, Claude Desktop, etc.).

- → Decidir opciones: `--headless` (por defecto abre ventana), `--isolated` (perfil en memoria, sin guardar sesiones), `--caps vision,pdf,devtools`, `--allowed-origins` y `--blocked-origins`.
- → No requiere API key ni cuenta.

USO

Una vez registrado, se le pide al agente por ejemplo *"abre esta página y rellena el formulario de contacto"*. Con el perfil persistente por defecto las sesiones iniciadas se conservan entre usos; con `--isolated` no.

- **Seguridad:** el README advierte que Playwright MCP *no es un límite de seguridad*, y que `--allowed-origins` tampoco lo es. Un agente con navegador puede leer páginas con tu sesión iniciada y actuar en ellas: usa perfiles aislados para sitios sensibles. Datos tomados del README del repo (la página playwright.dev no se pudo abrir desde el entorno donde se escribió esto); no instalé ni ejecuté la herramienta.

Firecrawl MCP

Paquete npm `firecrawl-mcp` v3.30.3 · repo firecrawl/firecrawl-mcp-server · MIT

SOLO MENCIONADA · SERVIDOR MCP

QUÉ HACE

Servidor MCP que conecta a un agente con Firecrawl para buscar, extraer e interactuar con la web y obtener contenido limpio para el contexto del agente. El README cuenta 27 herramientas con el perfil completo. Las principales:

HERRAMIENTA	CUÁNDO USARLA
`firecrawl_scrape`	Tienes una URL y quieres su contenido en Markdown o en JSON con un esquema.
`firecrawl_map`	Descubrir las URLs de un sitio sin traer su contenido.
`firecrawl_crawl`	Contenido de muchas páginas de un sitio, acotado con `limit`, rutas y profundidad.
`firecrawl_search`	Partir de una consulta y obtener resultados web, con contenido opcional.

`firecrawl_interact`	Páginas que necesitan clic, escritura o navegación antes de leerse.
`firecrawl_monitor_*`	Revisar una página de forma recurrente con diferencias y alertas.
`firecrawl_credit_usage`	Créditos restantes y consumo mensual.

INSTALACIÓN (SOLO REFERENCIA, NO INSTALADA)

```
# servidor remoto alojado por Firecrawl (nivel gratuito sin clave: scrape, sea
https://mcp.firecrawl.dev/v2/mcp

# local con npx
env FIRECRAWL_API_KEY=fc-TU_CLAVE npx -y firecrawl-mcp

# o global
npm install -g firecrawl-mcp
```

↗ Repositorio en GitHub ↗ Guía de Firecrawl ↗ npm

QUÉ FALTA PARA PODER INSTALARLA

- **Node.js 22 o superior** para la versión local (`engines` del paquete). El manual indica Node v24.18.0, que cumple.
- **Una API key de Firecrawl** (`fc-…`), que se crea en `firecrawl.dev/app/api-keys` , o conectar por OAuth con `https://mcp.firecrawl.dev/v2/mcp-oauth` . Sin clave solo funcionan `scrape` , `search` y `parse` , con límites; `crawl` , `map` y `agent` la exigen.
- Las llamadas consumen **créditos** de tu cuenta de Firecrawl; conviene revisar su plan y límites antes de dejar a un agente rastreando sitios.
- Guardar la clave en la configuración segura del cliente, nunca en la URL ni en el chat.
- Opcional: `FIRECRAWL_API_URL` si usas una instancia propia (autohospedada).

USO

Ejemplos de pedidos: *"extrae los precios de esta página como JSON"*, *"mapea las URLs de este sitio"* o *"busca documentación sobre X y dame el contenido"*.

- **Ojo:** el contenido que trae es texto de terceros que el agente leerá como contexto; una página maliciosa puede intentar darle instrucciones. Trata lo extraído como datos y no como órdenes, y respeta los términos y el `robots.txt` de los sitios. Datos tomados del README del repo (docs.firecrawl.dev no se pudo abrir desde el entorno donde se escribió esto); no instalé ni ejecuté la herramienta.

Perplexity MCP

Paquete npm `@perplexity-ai/mcp-server` v1.4.0 · repo perplexityai/modelcontextprotocol

SOLO MENCIONADA · SERVIDOR MCP

QUÉ HACE

Servidor MCP oficial de la plataforma API de Perplexity. Da a un asistente búsqueda web en tiempo real, razonamiento e investigación a través de la Search API y la Agent API de Perplexity. Herramientas:

HERRAMIENTA	QUÉ HACE
`perplexity_search`	Búsqueda web directa con resultados ordenados y metadatos; admite filtros de recencia y de dominio.
`perplexity_ask`	Respuesta conversacional con búsqueda en tiempo real (preset `fast`), para preguntas rápidas.
`perplexity_research`	Investigación profunda (preset `high`); puede tardar minutos.
`perplexity_reason`	Razonamiento para tareas analíticas complejas (preset `medium`).

INSTALACIÓN (SOLO REFERENCIA, NO INSTALADA)

```
# servidor remoto de Perplexity: nada que instalar
claude mcp add --transport http perplexity https://api.perplexity.ai/mcp \
  --header "Authorization: Bearer TU_CLAVE"

# local con npx
claude mcp add perplexity --env PERPLEXITY_API_KEY="tu_clave" -- npx -y @perp
```

↗ Repositorio en GitHub ↗ Guía de Perplexity ↗ npm

QUÉ FALTA PARA PODER INSTALARLA

- → **Una API key de Perplexity**, que se obtiene en el portal de la API (`console.perplexity.ai`). Es una cuenta de la plataforma API, distinta de una suscripción de la app; no verifiqué sus precios ni si hay crédito gratuito.
- → **Node.js 18 o superior** si se usa la versión local (el manual indica Node v24.18.0). La versión remota no necesita Node.
- → **Un cliente MCP** donde registrarlo (Claude Code, Codex, Cursor, VS Code, Claude Desktop, Kiro, Windsurf).
- → Opcional: `PERPLEXITY_TIMEOUT_MS` para investigaciones largas, `PERPLEXITY_PROXY` si estás tras un proxy corporativo y Docker si prefieres el modo HTTP autohospedado.
- → Guardar la clave como variable de entorno o secreto del cliente, no en el repositorio.

USO

Ejemplos de pedidos: *"busca las novedades de X de esta semana"* o *"investiga a fondo X y dame un informe con fuentes"*. Cada llamada usa la API de Perplexity con tu clave.

- Datos tomados del README del repo (docs.perplexity.ai no se pudo abrir desde el entorno donde se escribió esto); no instalé ni ejecuté la herramienta.

claude-video · /watch

`~/.claude/skills/watch/` · repo `bradautomates/claude-video` · v0.3.2 · MIT

INSTALADA EN EL CONTENEDOR DE LA SESIÓN

QUÉ HACE

Skill `watch` que le da a un agente evidencia de video: una URL o un archivo local se convierte en fotogramas con marcas de tiempo y una transcripción. Descarga con `yt-dlp`, extrae fotogramas con `ffmpeg` y lee los subtítulos nativos primero. Si no hay subtítulos, puede usar WhisperX local o transcripción en la nube (Groq u OpenAI). Con una clave `GEMINI_API_KEY`, el modelo de video de Google ve el video completo y el agente recibe su respuesta con marcas de tiempo.

Funciona en Claude Code (escritorio, VS Code o terminal) y en otros hosts de Agent Skills como Codex. **No funciona en Claude Chat ni en Cowork.**

INSTALACIÓN

```
# Claude Code, dentro de la sesión (método del README)
/plugin marketplace add bradautomates/claude-video
/plugin install watch@claude-video

# otros agentes
npx skills add bradautomates/claude-video -g --skill watch
```

- **Estado:** instalada el 18 oct 2026 **en el contenedor de nube de esta sesión**, no en tu PC con Windows. Copié `skills/watch` a `~/.claude/skills/watch/` (sin el plugin ni su hook de inicio, para no ejecutar el instalador de un tercero) e instalé `yt-dlp` 2026.08.19 con `pip --user`. Ya estaban `ffmpeg` 6.1.1 y Python 3.13.16. La skill aparece en la lista de skills de la sesión, pero **no ejecuté** su comprobación de dependencias (`setup.py`): el sistema de permisos bloqueó esa ejecución y no la forcé. El contenedor es efímero: al terminar la sesión se pierde. Para tu PC hay que repetir la instalación allí.

↗ Repositorio en GitHub

QUÉ NECESITA EN TU EQUIPO

- Python 3.10 o superior, `ffmpeg` / `ffprobe` y el `yt-dlp` más reciente (la skill solo admite la última versión). Para YouTube también un entorno JavaScript compatible (Deno).
- Windows: `winget install --id Gyan.FFmpeg --exact`, `winget install --id yt-dlp.yt-dlp --exact` y `winget install --id DenoLand.Deno --exact`.
- Opcional según el motor: `GEMINI_API_KEY` (clave gratuita de Google AI Studio), `GROQ_API_KEY`, `OPENAI_API_KEY`, o WhisperX local (al menos 3 GB de disco y 8 GB de RAM). Las claves van en `~/.config/watch/.env`, no en el chat ni en el repo.

USO

```
Use the watch skill on example.mp4. Summarize what is visible, with timestamps

/watch:watch video.mp4 --start 2:15 --end 2:45
/watch:watch video.mp4 --detail efficient --max-frames 30
/watch:watch https://example.com/video --engine local
```

Niveles de detalle: `transcript`, `efficient` (hasta 50 fotogramas), `balanced` (hasta 100, por escenas) y `token-burner` (sin tope).

- **Privacidad:** con el motor `local` no se contacta a Google. Con Gemini, el video se sube a la API de Google y luego se borra. Con Groq u OpenAI se envía el audio. Las

opciones `--cookies` y `--cookies-from-browser` dan acceso a sesiones iniciadas: úsalas solo si hace falta. Datos tomados del README y del código de la skill.

21st MCP

Servidor HTTP `https://21st.dev/api/mcp` · antes "Magic MCP" · repo 21st-dev/magic-mcp

REGISTRADO EN EL CONTENEDOR DE LA SESIÓN

QUÉ HACE

Servidor MCP de 21st.dev para trabajar con componentes de interfaz desde el agente. Reemplaza al antiguo Magic MCP (`@21st-dev/magic`). Según el README del repo, expone búsqueda en el catálogo de componentes, temas y plantillas (`search` , `get_component`), inspiración, búsqueda de logos (`search_logo`), marcadores, bibliotecas de equipo y, si la cuenta tiene acceso de IA, generación de UI con variantes (`generate` , `iterate_generation`). `get_usage` indica si la generación está habilitada.

INSTALACIÓN

```
# asistente oficial
npx @21st-dev/cli@latest init --client claude   # o cursor | vscode | windsur

# manual, como servidor HTTP en Claude Code
claude mcp add --scope user --transport http 21st https://21st.dev/api/mcp \
  --header "x-api-key: TU_CLAVE_21ST"
```

- **Estado:** registrado el 18 oct 2026 con `claude mcp add --scope user` en el contenedor de nube de esta sesión (queda en el `~/.claude.json` del contenedor, no en el repositorio ni en este manual). Es efímero y no está en tu PC. **No verifiqué la conexión**: las herramientas aparecen solo en una sesión nueva, y el servidor puede rechazar la clave si no tiene acceso. La clave no se escribe aquí.

↗ Repositorio en GitHub ↗ Claves y configuración ↗ CLI en npm

QUÉ NECESITA

→ Una clave de API de 21st.dev, que se genera en `21st.dev/mcp` . Las claves del antiguo Magic dejaron de funcionar.

- Node.js solo si usas el asistente `npx @21st-dev/cli` o el proxy `@21st-dev/magic`; el servidor HTTP directo no lo necesita.
- Sin acceso de IA solo funcionan la búsqueda y la descarga de componentes; `generate` y `iterate_generation` no aparecen.
- El repo también se ofrece como plugin con una skill de UI: `claude plugin marketplace add 21st-dev/magic-mcp` y luego `/plugin install 21st`, con la clave en la variable `API_KEY_21ST`.

- **Seguridad de la clave:** una clave `21st_sk_…` es un secreto que da acceso a tu cuenta y su cuota. Si se pega en un chat o en un documento, hay que considerarla expuesta y regenerarla en `21st.dev/mcp`. No la guardes en archivos que se suban a Git: usa el almacén de secretos del cliente o una variable de entorno. Datos tomados del README del repo y de una búsqueda web; 21st.dev no se pudo abrir desde el entorno donde se escribió esto.

system_prompts_leaks

Repo asgeirtj/system_prompts_leaks · CC0 · colección de referencia

SOLO REFERENCIA · NADA QUE INSTALAR

QUÉ ES

Repositorio de Markdown que reúne lo que se presentan como los prompts de sistema completos de los chatbots y productos de IA más usados. Está organizado por empresa: Anthropic, OpenAI, Google, Meta, Microsoft, Mistral, xAI, DeepSeek, Qwen, Kimi, GLM, Perplexity, Notion, Cursor y otros. La carpeta de Anthropic tiene archivos por modelo y por producto (Claude Code, Cowork, Claude in Chrome, Claude para Excel y Word, entre otros).

No es una skill, una herramienta ni un servidor MCP: es material de lectura. No hay nada que instalar; para consultarlo basta con abrirlo en GitHub o clonarlo.

PARA QUÉ SIRVE

- Ver cómo estructuran las instrucciones, las reglas de herramientas y el estilo de respuesta los distintos productos, y aprender de ello al escribir las tuyas (por ejemplo, el `SKILL.md` de una skill o un prompt de agente).
- Comparar el comportamiento prometido de un producto con el real.
- Investigar cómo se reparten las instrucciones entre prompt de sistema, herramientas y recordatorios.

↗ Repositorio en GitHub

- **Ten en cuenta:** el contenido lo aporta la comunidad y no tiene verificación oficial; un archivo puede estar incompleto, desactualizado o ser inexacto, y las empresas cambian sus prompts sin avisar. Algunos productos tienen términos de uso que restringen extraer o redistribuir sus instrucciones. Úsalo para aprender, no como especificación de cómo se comporta un producto. No copies estos prompts a tus agentes sin revisarlos: pueden traer instrucciones ajenas a tu caso. Datos tomados del README y de la estructura de carpetas del repo; no verifiqué la autenticidad de ningún prompt.

gstack

SUITE · 57 SKILLS

~/.claude/skills/gstack/ · repo garrytan/gstack · v1.91.126

QUÉ HACE

Convierte a Claude Code en un equipo de ingeniería virtual: un CEO que repiensa el producto, un eng manager que fija la arquitectura, un diseñador que caza el "AI slop", un revisor que encuentra bugs de producción, un QA que abre un navegador real, un oficial de seguridad y un release engineer que envía el PR. Son comandos slash en Markdown, con licencia MIT, creados por Garry Tan (presidente y CEO de Y Combinator).

No es una colección de herramientas sueltas sino un proceso, y las skills corren en el orden de un sprint, cada una alimentando a la siguiente:

```
Think → Plan → Build → Review → Test → Ship → Reflect
```

INSTALACIÓN

```
# desde Claude Code, o en terminal
git clone --single-branch --depth 1 https://github.com/garrytan/gstack.git \
  ~/.claude/skills/gstack
cd ~/.claude/skills/gstack && ./setup

# modo equipo (desde dentro de tu repo): auto-actualización para todos
(cd ~/.claude/skills/gstack && ./setup --team) \
  && ~/.claude/skills/gstack/bin/gstack-team-init required
```

↗ Repositorio en GitHub

REQUISITOS

- → Claude Code y Git.
- → Bun 1.4.2+ (versión probada; `./setup` rechaza Bun anterior a 1.3.3). Node.js solo en Windows.
- → Recomendado en macOS: el navegador Aside; sin él, `./setup` construye el navegador propio de gstack.
- → `/cso` necesita además un Bun con las cuatro opciones `--no-compile-autoload-*` y un compilador nativo; si faltan, reporta *not assessed* y el resto sigue funcionando.

PRIMEROS PASOS

1. `/office-hours`: describe lo que estás construyendo.
2. `/plan-ceo-review` sobre cualquier idea de feature.
3. `/review` sobre cualquier rama con cambios.
4. `/qa` sobre tu URL de staging o una API local aislada.

QUÉ REVISIÓN USAR

CONSTRUYES PARA…	ANTES DEL CÓDIGO	AUDITORÍA EN VIVO
Usuarios finales (UI, web, móvil)	`/plan-design-review`	`/design-review`
Desarrolladores (API, CLI, SDK, docs)	`/plan-devex-review`	`/devex-review`
Arquitectura (datos, rendimiento, tests)	`/plan-eng-review`	`/review`
Todo lo anterior	`/autoplan`	—

PENSAR Y PLANEAR

SKILL	QUÉ HACE
`/office-hours`	YC Office Hours: seis preguntas que replantean el producto antes de escribir código. Su documento de diseño alimenta a las demás skills.
`/plan-ceo-review`	Modo CEO/fundador: repiensa el problema. Cuatro modos: Expansion, Selective Expansion, Hold Scope y Reduction.

`/plan-eng-review`	Modo eng manager: fija arquitectura, flujo de datos, casos límite y tests.
`/plan-design-review`	Diseñador senior: puntúa cada dimensión de 0 a 10, detecta "AI slop" y edita el plan.
`/plan-devex-review`	Revisión interactiva de developer experience: personas, TTHW frente a la competencia y puntos de fricción.
`/design-consultation`	Construye un sistema de diseño completo y escribe `DESIGN.md`.
`/autoplan`	Corre CEO → diseño → DX → ingeniería en secuencia y solo te consulta las decisiones de gusto.
`/spec`	Convierte una intención vaga en un spec ejecutable en cinco fases.
`/eval-plan`	Preregistra una evaluación de pago antes de la primera llamada: barras, brazos, reglas de parada y piloto de tres ítems.

CONSTRUIR Y NAVEGAR

SKILL	QUÉ HACE
`/design-shotgun`	Genera 4–6 variantes de mockup, abre un tablero de comparación y recoge tu feedback.
`/design-html`	Convierte un mockup en HTML de producción; detecta React, Svelte o Vue.
`/browse`	Navegador para el agente: usa Aside si está, si no el Chromium propio de gstack.
`/scrape`	Extrae datos estructurados de una página; `/skillify` lo convierte en una browser-skill permanente.
`/skillify`	Convierte el último `/scrape` exitoso en una skill de navegador.
`/open-gstack-browser`	Lanza el navegador de gstack con sidebar, stealth anti-bot y enrutamiento automático de modelo.
`/setup-browser-cookies`	Copia cookies seleccionadas de tu navegador al navegador de gstack (solo en la vía de respaldo).

`/connect-chrome`	Conecta el navegador de gstack con tu Chrome.
`/pair-agent`	Comparte el navegador con otro agente de IA (OpenClaw, Codex, Cursor...) con tokens acotados y aislamiento por pestaña.

REVISAR Y PROBAR

SKILL	QUÉ HACE
`/review`	Staff engineer: busca los bugs que pasan CI y fallan en producción; autocorrige lo obvio.
`/investigate`	Debugging de causa raíz: sin arreglos sin investigación; se detiene tras 3 intentos fallidos.
`/design-review`	Misma auditoría que el plan, pero corrige: commits atómicos y capturas antes/después.
`/devex-review`	Auditoría en vivo de developer experience; prueba el onboarding de verdad.
`/cso`	Chief Security Officer: auditoría con modelo de la aplicación, hallazgos sustentados y cobertura explícita.
`/test-audit`	Barre los tests que cuestan más de lo que protegen; solo reporta salvo que apruebes un lote.
`/deslop-shared-libs`	Busca extracciones de código compartido que valgan la pena; solo recomendaciones.
`/health`	Dashboard de calidad de código.
`/qa`	QA Lead: explora navegador, API, CLI, jobs y webhooks; reproduce, escribe regresión, corrige y reverifica.
`/qa-only`	QA de solo reporte, con evidencia reproducible y sin tocar código.
`/codex`	Segunda opinión independiente desde OpenAI Codex CLI (review, challenge, consult).
`/claude-code`	Segunda opinión desde Claude Code; disponible en todo arnés menos Claude Code.

ENVIAR Y REFLEXIONAR

SKILL	QUÉ HACE
`/ship`	Release engineer: sincroniza main, corre tests, audita cobertura y docs, verifica, hace push y abre el PR.
`/setup-deploy`	Configuración única para `/land-and-deploy`: plataforma, URL de producción y comandos.
`/land-and-deploy`	Mergea el PR, espera CI y deploy, y verifica la salud en producción.
`/landing-report`	Dashboard de solo lectura de la cola de ship por workspace.
`/canary`	Monitoreo post-deploy: errores de consola, regresiones de rendimiento y fallas de página.
`/benchmark`	Base de tiempos de carga, Core Web Vitals y tamaños de recursos; compara antes/después.
`/benchmark-models`	Benchmark cruzado entre modelos para las skills de gstack.
`/document-release`	Audita el comportamiento cambiado contra la documentación en cada ship.
`/document-generate`	Genera documentación faltante con el marco Diátaxis.
`/retro`	Retrospectiva semanal del equipo; `/retro global` abarca todos tus proyectos y herramientas de IA.

SEGURIDAD Y UTILIDADES

SKILL	QUÉ HACE
`/careful`	Avisa antes de comandos destructivos (r -rf, DROP TABLE, force-push).
`/freeze · /unfreeze`	Restringe las ediciones a un directorio · qu la restricción.
`/guard`	`/careful` + `/freeze` en u

	solo comando
`/context-save · /context-restore`	Guarda y restaura el contexto de trabajo.
`/learn`	Revisa, busca poda lo que gstack aprend entre sesione
`/make-pdf`	Markdown a PDF de calida de publicació también `--to html` y `--to docx`.
`/diagram`	Descripción e inglés a diagrama editable: fuen mermaid, `.excalidraw` SVG/PNG.
`/setup-gbrain · /sync-gbrain`	Instala gbrain (PGLite o Supabase) y mantiene indexado el código del rep
`/gstack-upgrade`	Actualiza gsta a la última versión.
`/ios-qa · /ios-fix · /ios-design-review · /ios-clean · /ios-sync`	Suite iOS: QA dispositivo rea corrector de bugs, auditorí HIG, limpieza resincronizaci del debug bridge.

- Datos tomados del README y de las carpetas del repo (v1.91.126); no ejecuté `./setup` ni verifiqué el estado de la instalación local. La versión instalada en tu equipo puede diferir. El README sugiere añadir a `CLAUDE.md` que use `/browse` de gstack para toda navegación web en lugar de las herramientas `mcp__claude-in-chrome__*` . La telemetría tiene su propia sección en el README: revísala antes de activarla.

ECC (Everything Claude Code)

SOLO REFERENCIA

Referencia · no instalado · paquete npm `ecc-universal` v2.2.3

QUÉ ES

Un "sistema operativo de agente" para Claude Code (con adaptadores limitados para Codex, Cursor, OpenCode, Gemini y otros). No es una sola skill: es un kit de agentes, skills, hooks y reglas con licencia MIT que impone un ciclo de ingeniería fijo en lugar de improvisar en cada prompt.

```
plan → test → implement → review → verify → remember → improve
```

QUÉ INCLUYE

71 agentes
Planificación, revisión por lenguaje, reparación de builds, seguridad, arquitectura, base de datos, E2E.

306 skills
TDD, investigación, seguridad, docs, frontend, datos, ML, operaciones, patrones de agentes.

97 comandos
Atajos heredados mientras el proyecto migra a un modelo basado en skills.

Hooks y memoria
Se disparan antes y después de cada herramienta: bloquean o avisan, guardan resúmenes de sesión y aprenden patrones ("instincts").

Reglas
Estándares siempre cargados que eliges por lenguaje o proyecto (Python, Go, Rust, React, Swift...).

AgentShield
Escáner de seguridad para prompts, hooks, MCP, permisos y secretos de tus archivos de agente.

INSTALACIÓN (SI DECIDES USARLO)

```
# guiado; requiere Node 18+, Git y Claude Code 2.1+
npx ecc-universal@2.2.3 setup

# o como plugin de Claude Code (elige un solo método, no los apiles)
ecc@ecc
```

↗ Repositorio en GitHub ↗ ecc.tools

- **Antes de instalar:** solapa con `claude-mem` (memoria), `gstack` (revisión, QA y envío) y `security-audit`, así que habría varios sistemas de hooks y memoria compitiendo, además de más contexto cargado en cada sesión. Los hooks ejecutan código local en cada acción: revisa el perfil de hooks antes de activarlo. Es un proyecto de un solo mantenedor; el núcleo es gratis y hay una versión de pago (ECC Pro, GitHub App para repos privados).

GESTIÓN DE ESTAS SKILLS

→ Todas se activan solas cuando el pedido coincide con su descripción, o al invocarlas con `/nombre-skill`.

→ ui-ux-pro-max y la suite gstack están instaladas a nivel de usuario (`~/.claude/skills/`): aplican a cualquier proyecto que abras en esta máquina. Para quitar una, borra su carpeta; para actualizarla, vuelve a clonar y sobrescribe.

→ claude-mem es distinto: es un **plugin** gestionado por Claude Code (`~/.claude/plugins/`), con un worker en segundo plano. Se administra con `npx claude-mem <comando>` o `/plugin uninstall claude-mem`, no borrando carpetas a mano.

→ Este documento refleja el estado al momento de generarlo — si instalas o quitas algo después, pide que se vuelva a actualizar.

Verificado en este PC (Windows) el 17 oct 2026

63 skills · ui-ux-pro-max (7) + gstack y otras (56) + claude-mem (plugin)